Atividade Prática 2

CONTEXTO:
Nesta atividade prática, você testará requisições **HTTP (GET, POST, PUT e DELETE)** utilizando a **Mangá Base API** localmente e o aplicativo **Bruno**.

Guia de instalação:
1. Baixar o Bruno (Se ainda não instalou): Acesse **https://www.usebruno.com/** e instale o app.
2. No terminal (na sua pasta de usuário) baixe o projeto da API com o comando:
git clone git@github.com:batistabjs/manga-base-api.git
3. No VS Code, abre a pasta **manga-base-api**
4. Ainda No VS Code, abra o terminal e execute os comandos:
npm install
npm start
A API rodará no endereço **http://localhost:8080**
5. No app Bruno, crie uma nova Collection chamada **Manga-API**

DICA 1: Onde consultar os dados já pré-cadastrados?
Onde consultar os dados pré-cadastrados: Para verificar os mangás e usuários já cadastrados no projeto, abra os arquivos JSON na pasta **/dados** (**dados/mangas.json** e **dados/usuarios.json**)
DICA 2: Dê ênfase à seção **"Exemplos de Uso"** no arquivo **README.md** do projeto; ela mostra exatamente como estruturar as URLs e os dados (JSON) das requisições.

DESAFIO:
Crie e salve as 20 requisições abaixo dentro da sua Collection no Bruno:

1. GET - Listar Mangá
Método: GET | URL: http://localhost:8080/api/v1/mangas
Objetivo: Retornar a lista completa de obras cadastradas (público).

2. GET - Buscar Mangá por ID
Método: GET | URL: http://localhost:8080/api/v1/mangas/1
Objetivo: Consultar os dados específicos do mangá com ID 1.

3. GET - Rota Específica por Tipo
Método: GET | URL: http://localhost:8080/api/v1/mangas/tipo/manhwa
Objetivo: Filtrar apenas as obras do tipo manhwa usando a rota dedicada.

4. GET - Filtrar Tipo por Query Param
Método: GET | URL: http://localhost:8080/api/v1/mangas?tipo=gibi
Objetivo: Filtrar a listagem principal passando o parâmetro tipo=gibi

5. GET - Rota Específica por Autor

Método: GET | URL: http://localhost:8080/api/v1/mangas/autor/Eiichiro%20Oda
Objetivo: Filtrar obras publicadas por um autor específico via endpoint dedicado.

6. GET - Rota Específica por Status
Método: GET | URL: http://localhost:8080/api/v1/mangas/status/em_andamento
Objetivo: Filtrar obras pelo status (em_andamento, completo, hiatus, cancelado).

7. GET - Rota Específica por Gênero
Método: GET | URL: http://localhost:8080/api/v1/mangas/genero/Ação
Objetivo: Buscar obras de um gênero específico.

8. GET - Rota Específica por Ano
Método: GET | URL: http://localhost:8080/api/v1/mangas/ano/1997
Objetivo: Retornar obras lançadas no ano informado.

9. GET - Busca Textual no Endpoint
Método: GET | URL: http://localhost:8080/api/v1/mangas/buscar?q=piece
Objetivo: Fazer uma busca por palavra-chave no endpoint de busca rápida.

10. GET - Busca Textual por Query Param
Método: GET | URL: http://localhost:8080/api/v1/mangas?busca=one
Objetivo: Fazer busca textual nos campos de título, autor e sinopse.

11. GET - Paginação e Ordenação
Método: GET | URL: http://localhost:8080/api/v1/mangas
Query Params: pagina=1, tamanho_pagina=3, ordenar_por=ano_lancamento, direcao_ordenacao=desc
Objetivo: Retornar apenas 3 itens da página 1 ordenados pelo ano de lançamento decrescente.

12. GET - Consultar Estatísticas da API
Método: GET | URL: http://localhost:8080/api/v1/mangas/estatisticas
Objetivo: Exibir dados agregados e métricas sobre a base de mangás.

13. POST - Cadastrar Novo Usuário (Público)
Método: POST | URL: http://localhost:8080/api/v1/auth/cadastro
Body (JSON):

```
{
  "nome": "Leitor Teste",
  "email": "leitor@mangabase.com",
  "senha": "123"
}
```

14. POST - Login e Obter Token JWT (Público)
Método: POST | URL: http://localhost:8080/api/v1/auth/login
Body (JSON): Use o usuário admin@mangabase.com (senha 123456) ou o criado no desafio 13.

```
{
  "email": "admin@mangabase.com",
  "senha": "123456"
}
```


15. GET - Validar Usuário Logado (Autenticado)
Método: POST | URL: http://localhost:8080/api/v1/auth/me
Auth (Bruno): Selecione Bearer Token e informe o token do Desafio 14.
Objetivo: Retornar os dados do usuário autenticado.

16. POST - Cadastrar Novo Mangá (Autenticado)
Método: POST | http://localhost:8080/api/v1/mangas
Auth (Bruno): Bearer Token
Body (JSON):


```
{
  "titulo": "Chainsaw Man",
  "tipo": "manga",
  "autor": "Tatsuki Fujimoto",
  "editora": "Shueisha",
  "sinopse": "Denji é um jovem que vive como caçador de demônios.",
  "ano_lancamento": 2018,
  "status": "em_andamento",
  "volumetoria": "16+ volumes",
  "genero": "Ação, Sobrenatural",
  "classificacao_etaria": "18",
  "url_capa": "https://example.com/chainsaw.jpg"
}
```


17. POST - Cadastrar Novo Manhwa ou Gibi (Autenticado)
Método: POST | URL: http://localhost:8080/api/v1/mangas
Auth (Bruno): Bearer Token
Body (JSON):


```
{
  "titulo": "Turma da Mônica Especial ",
  "tipo": "gibi",
  "autor": "Mauricio de Sousa",
  "sinopse": "Histórias e aventuras no Bairro do Limoeiro.",
  "ano_lancamento": 1970,
  "volumetoria": "Vários volumes",
  "genero": "Comédia, Infantil",
  "classificacao_etaria": "L",
  "url_capa": "https://example.com/monica.jpg"
}
```

18. PUT - Atualizar Mangá Existente (Autenticado)

Método: PUT | URL: http://localhost:8080/api/v1/mangas/1 (ou o ID do item criado)

Auth (Bruno): Bearer Token

Body (JSON):

```
{
  "status": "completo",
  "volumetoria": "34 volumes finalizados",
}
```

19. DELETE - Remover Mangá (Autenticado)

Método: DELETE | URL: http://localhost:8080/api/v1/mangas/1 (ou o ID do item criado)

Auth (Bruno): Bearer Token

Objetivo: Deletar o registro selecionado e verificar a confirmação (200 OK).

20. DELETE/POST - Teste de Segurança Sem Token (Erro 401)

Método: DELETE | URL: http://localhost:8080/api/v1/mangas/1

Auth (Bruno): No Auth (sem enviar header Authorization).

Objetivo: Comprovar que a API bloqueia a requisição e retorna o código de status 401 Unauthorized

ENTREGA:

O professor avaliará no seu computador o sucesso (ou não) de cada uma das atividades através de:

- Verificação da aplicação em execução no terminal (http://localhost:8080).
- Apresentação da Collection no Bruno com as 20 requisições devidamente salvas e testadas.
- Demonstração prática do envio das requisições usando o token nas rotas protegidas (POST, PUT e DELETE) e do teste de segurança 401 Unauthorized